# A Reference Architecture for Intellectual Capital Platforms

*The minimum infrastructure required to operationalize an organization's truth-finding methodology*

---

Interface-first specification, with reference implementations

Capture → Reasoning Engine → Repository

*This document specifies the bare-bones infrastructure for what we will call an **Intellectual Capital Platform** (ICP): a system that turns an organization's stream of unstructured deliberation (meetings, writing, research, conversation) into a navigable, queryable, validated body of knowledge whose conclusions are traceable to evidence and whose methods are themselves auditable artifacts. The architecture is presented in three layers — Capture, Reasoning Engine, Repository — with each layer specified first as a set of interface contracts and then as a set of recommended implementations. The document is intended for an engineering organization that wishes to build the system rather than buy it. Adopters who care only about the design pattern may stop after Section 2.*

Version 1.0 | 2026-05-18

# Contents

# 1. Introduction: What an Intellectual Capital Platform is

## 1.1 The problem

Every organization that does intellectual work — a research firm, an investment house, a think tank, a publisher, a consultancy, a policy unit, a media operation — produces *intellectual capital* as its primary output: arguments, principles, predictions, frameworks, syntheses, taxonomies. In nearly every such organization, this capital sits in one of three places: in the heads of a small number of senior people, in a sprawl of unsearchable documents (decks, memos, emails, transcripts), or in a knowledge-base tool that was never designed for argumentation and so degrades into a glorified file cabinet.

The cost of this is rarely measured but always paid. New hires repeat work that was done five years ago because no one remembers. Contradictions between two analysts go undetected for months because no system juxtaposes their claims. Predictions are made and then quietly forgotten, so the firm has no honest track record. Methods are not distinguishable from the people who happen to deploy them, so when those people leave, the methods leave with them.

An *Intellectual Capital Platform* is the infrastructure that makes the firm's intellectual output a first-class persisted object, structured at the level of atomic claims and their relationships, validated against an explicit set of coherence and rigor checks, and traceable in both directions: from raw source up to published conclusion, and from a published conclusion back down to the sentence that originated it.

## 1.2 What an ICP is not

It is worth being precise about what is being specified, since the term "knowledge platform" has been claimed by every wiki, every vector database, and every retrieval-augmented chatbot.

- An ICP is not a wiki. Wikis store text. An ICP stores atomic propositions with typed relationships and validation history.
- An ICP is not a vector database with a chat layer on top. A vector database treats every passage as independent; an ICP preserves logical relationships — supports, contradicts, refines, instantiates — without which contradiction detection and provenance traversal are impossible.
- An ICP is not an LLM agent harness. LLM inference is a tool the platform uses, not the platform itself. The platform's correctness does not depend on the LLM being correct; it depends on the platform being able to detect when the LLM is wrong.
- An ICP is not a CRM or a project tracker. It tracks claims, not tasks.

## 1.3 The seven invariants

The architecture in this document is opinionated about a small number of properties that any system claiming to be an ICP must satisfy. The rest is implementation choice.

**Invariant 1 — Atomicity**

The unit of storage and analysis is the atomic proposition, not the paragraph, document, or summary. Summaries destroy the logical structure (subject, predicate, modality, quantification) on which every downstream check depends.

**Invariant 2 — Provenance is non-optional**

Every atomic claim is anchored to a specific span of a specific source (byte offsets, timestamp ranges, page-and-line). A claim without a citation is rejected at ingestion. Provenance is bidirectional: forward (source → conclusion) and reverse (conclusion → source).

**Invariant 3 — Multi-layer validation**

No single signal — not embedding similarity, not natural-language inference, not LLM judgment — is sufficient to validate coherence. The platform composes multiple independently-failing checks and aggregates them with explicit weighting. A claim's coherence score is a vector, not a scalar.

**Invariant 4 — Graph, not vector store**

The persisted knowledge structure is a typed graph: nodes are claims and principles, edges are typed relationships (*supports*, *contradicts*, *refines*, *instantiates*, *depends-on*, *generalizes*). A vector index is a secondary structure layered on top for retrieval, not the primary store.

**Invariant 5 — Temporal indexing**

Every node and edge carries a timestamp and a conviction score. The same principle stated in week 1 and week 50 strengthens its conviction; the same principle later contradicted records a drift. Knowledge ages; the platform must model decay, not pretend everything is equally fresh.

**Invariant 6 — Adversarial-by-construction**

The platform generates counter-claims, runs red-team checks, and submits its conclusions to multi-perspective peer review as part of the standard pipeline, not as an after-the-fact audit. A conclusion that has never been attacked is treated as unvalidated regardless of its coherence score.

**Invariant 7 — Methods are first-class**

Every method by which the platform extracts, scores, or infers — the claim extractor, the NLI scorer, the geometric contradiction detector, the rigor gate — is a first-class persisted object with a declared interface, a versioned implementation, a track record, and a drift monitor. Methods are themselves auditable; an organization's intellectual capital is as much its methods as its claims.

**RATIONALE**

The seventh invariant is the one most often skipped, and skipping it is what reduces a knowledge platform to a content management system. The product of an ICP is not the body of claims it stores; it is the methodology by which those claims are validated. An organization that adopts this template adopts a commitment to operationalizing its truth-finding methodology, not merely its truth claims.

### 1.4 Audience and scope

This document is written for a small engineering team (3–6 people for the minimum-viable build, 8–15 for a production deployment) who have been asked to build the platform their organization needs. It assumes the reader is comfortable with distributed systems, modern web development, basic information retrieval, and the practical realities of operating an LLM-dependent service. It does not assume familiarity with the specific implementation that motivated the abstraction.

## 2. The system at a glance

### 2.1 Three layers, one ledger

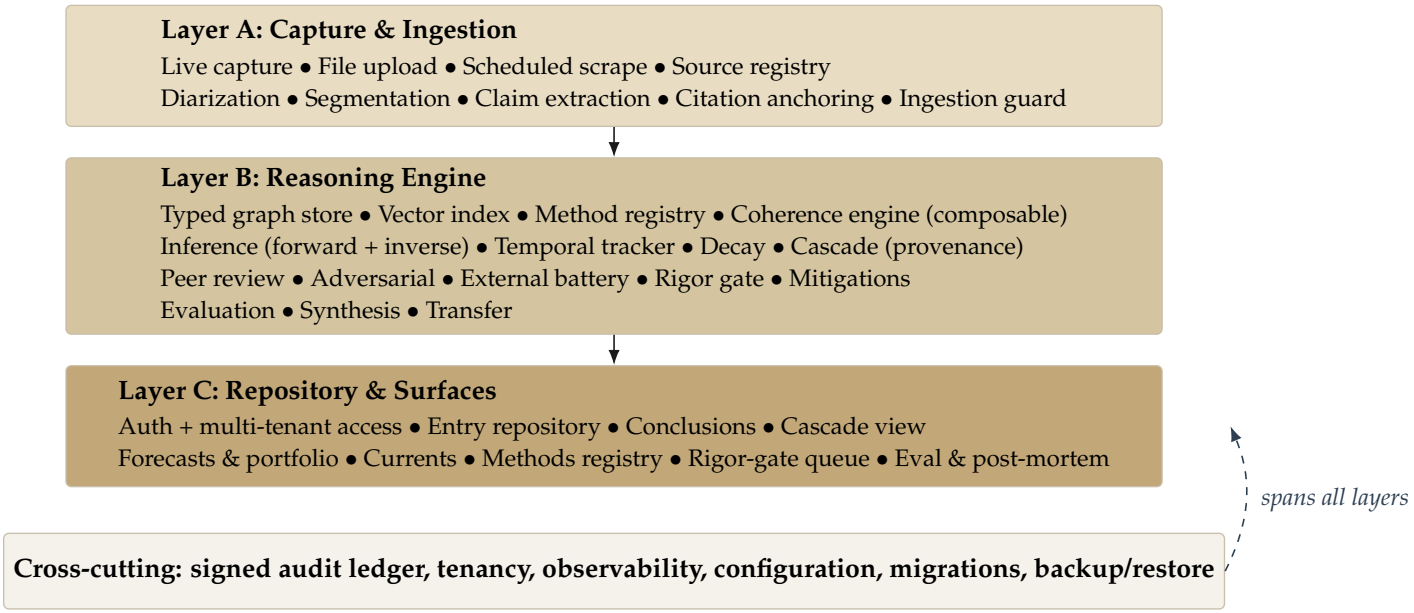


Figure 1: The three-layer anatomy. Data flows downward (capture → reasoning → surfaces). Every state-changing operation in any layer emits an event to the signed audit ledger.

### 2.2 Pipeline view

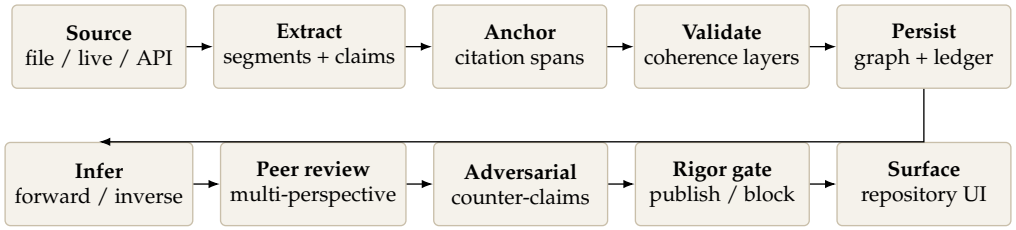


Figure 2: End-to-end dataflow. The first row is ingestion; the second is reasoning and publication. The pipeline is not strictly linear in production — peer review and adversarial generation often feed back into the graph as new claims — but this is the canonical happy path.

### 2.3 Three crisp questions the platform must answer

A useful sanity check for any ICP design is whether it can answer each of the following questions in bounded time, with a defensible audit trail.

1. **Provenance.** "Show me every primary source that the conclusion $C$ descends from, and the chain of intermediate claims, ranked by how load-bearing each is."
2. **Contradiction.** "Of everything our firm has on the record, what most clearly contradicts the claim that $P$, and what is the relative conviction of the two positions?"

3. **Calibration.** "Of the predictions our firm made between dates $t_0$ and $t_1$ at confidence $\geq c$, what fraction came true, sliced by topic and by method that generated them?"

If a candidate architecture cannot answer all three without manual analysis, it is not an ICP — it is a content management system with extra steps.

## 3. Layer A — Capture & Ingestion

### 3.1 Purpose and boundary

Layer A converts unstructured inputs — spoken conversations, written documents, scraped web content, API events — into structured, citation-anchored atomic claims that the Reasoning Engine can consume. Its responsibility ends the moment a validated `Claim` (with its anchored `CitationSpan`) is committed to the store. It is deliberately ignorant of what the Reasoning Engine will do with the claims.

The hardest design discipline in Layer A is restraint. Adopters often try to push reasoning into ingestion ("while we're parsing, let's also score coherence"). This couples layers that should remain orthogonal and makes both untestable. Layer A's job is to produce *faithful atomic claims with anchored provenance*. That is all.

### 3.2 Interface contracts

Listing 1: Capture-side contracts.

```
record Source {
   id: SourceId
   kind: enum { live_capture, file_upload, scheduled_scrape, api_push }
   medium: enum { audio, video, text, pdf, html, structured_json }
   origin_uri: URI
   captured_at: Timestamp
   captured_by: UserId
   sha256: HashHex // immutable content fingerprint
   metadata: Map<String, Any> // arbitrary, e.g. attendees, channel
   invariant: sha256 uniquely identifies the binary content; re-uploads de-dup.
}

record CaptureSession { // for live audio / live conversation
   id: SessionId
   started_at: Timestamp
   participants: List<SpeakerId>
   consent_state: enum { silent, passive, conversational, tutor }
   source_id: SourceId // points to the immutable audio artifact
   invariant: consent_state is set before any audio is persisted.
}

record Segment { // a temporal/positional slice of a Source
   id: SegmentId
   source_id: SourceId
   span: Span // byte range, time range, or page+line
   speaker: SpeakerId? // for diarized audio
   text: String // canonical text form
   embedding: Vector? // optional, populated by Reasoning Engine
}

record Claim { // the atomic unit the rest of the system uses
   id: ClaimId
   source_id: SourceId
   segment_ids: List<SegmentId>
   text: String // a single proposition, ideally < 30 tokens
   modality: enum { assertion, prediction, question, counter, hypothetical }
   speaker: SpeakerId?
   extracted_at: Timestamp
```

```
   extraction_method: MethodId // see Section 4.3
   confidence: Float ∈[0,1] // extractor's confidence the claim is faithful
   citation: CitationSpan // required, not nullable

   precondition: text is an atomic proposition (no conjunction across subjects)
   precondition: citation resolves to a non-empty span of source_id
}

record CitationSpan {
   source_id: SourceId
   locator: union { // medium-specific locator
      byte_range: (start: Int, end: Int),
      time_range: (start: Seconds, end: Seconds),
      page_line: (page: Int, line_start: Int, line_end: Int),
      dom_xpath: String
   }
}
```

## 3.3 Components

### *3.3.1 Capture endpoints*

A production ICP needs at minimum four capture modes; the same internal interfaces serve all four.

**Live capture (desktop or browser).**
A small client (PyQt6 / Electron / Tauri / native browser `MediaRecorder`) records audio with explicit consent state and uploads the artifact plus a JSONL session manifest. The client is the only component that needs to run on the user's machine; everything downstream runs server-side.

**File upload.**
Web upload for PDFs, text, audio, video. Large files chunked and resumable.

**Scheduled scrape.**
Cron-style ingestion from feeds the organization cares about — X / Twitter, RSS, Substack, podcast feeds, prediction-market APIs. Each source is registered with explicit rate limits and rotation policy.

**API push.**
Webhook endpoint for third-party tools (transcription services, CRM, document management) that already produce structured artifacts.

### *3.3.2 Source registry*

Every capture, regardless of channel, registers itself with a single `SourceRegistry`. The registry's job is content fingerprinting (so the same PDF re-uploaded twice is one source, not two), origin tracking, consent metadata, and producing the immutable `Source` record on which all downstream claims hang.

**Invariant: every claim has exactly one source**

A claim may be supported by multiple sources later (via evidence relationships in the graph), but at extraction time it descends from exactly one source. This makes provenance tractable.

#### *3.3.3 Ingestion guard*

The ingestion guard is a small, deliberately paranoid module that sits between the capture endpoints and the extractor. Its responsibilities:

- Reject sources whose hash matches a known-malicious or known-spam fingerprint.
- Reject text that contains obvious prompt-injection patterns ("ignore previous instructions", etc.) targeted at the downstream LLM extractor. The guard is a pattern detector, not an LLM — it must not itself be susceptible to injection.
- Strip or escape control sequences that some PDF / HTML extractors emit in ways that confuse downstream parsing.
- Enforce per-tenant ingestion quotas and per-source rate limits.
- Emit a `ingestion_attempted` ledger event for every input regardless of acceptance.

#### *3.3.4 Diarization & segmentation*

For audio / video sources, an ASR step transcribes and a diarization step assigns speaker labels. For text / PDF sources, a segmenter splits on sentence boundaries with semantic-paragraph awareness. The output of either path is a list of `Segment` records with optional `speaker` attribution.

The platform should support *two* ASR backends and select at runtime: a local model (faster-whisper or equivalent) for privacy-sensitive deployments, and a cloud API (OpenAI Whisper, AssemblyAI) as fallback and quality benchmark. Diarization should support speaker enrollment so that recurring participants are stably identified across sessions.

#### *3.3.5 Claim extractor*

The extractor is the most algorithmically interesting component in Layer A. It consumes `Segment`s and emits `Claim`s. The contract is that every emitted claim is (a) atomic — a single proposition with a single subject and predicate, (b) faithful — semantically entailed by the segment, not invented, and (c) anchored — carries a non-trivial `CitationSpan`.

Two orthogonal extractors should be implemented:

1. **LLM extractor.** A frontier LLM (Claude, GPT-4-class) prompted with a strict JSON schema, the segment text, and the speaker context. The output is validated against the schema and against a faithfulness check (next paragraph) before being accepted.
2. **Linguistic fallback.** A rule-based extractor built on spaCy or equivalent: dependency parsing to identify subject-predicate-object triples, modality detection, negation handling. This exists for two reasons: as a sanity check on the LLM, and as the only available extractor when the LLM is unavailable, rate-limited, or refused for the document.

A *faithfulness check* runs an NLI model (cross-encoder, e.g. DeBERTa-v3) on each (segment, claim) pair and rejects any claim that does not score entailment above a configured threshold. This is

cheap, fast, and catches the most common LLM extractor failure (paraphrase drift).

#### *3.3.6 Citation anchor*

Each `Claim` must point back to a non-trivial span of its source. The anchor module's job is, given a claim and the segments it was extracted from, to compute the tightest defensible span. For audio: the time range covering the segments. For PDF / text: the byte range. For HTML: the DOM xpath. The anchor module also runs a verification pass: it reads the anchored span back from the source and confirms that the claim's text is plausibly entailed by it. This catches the silent failure mode where the extractor hallucinates a claim that has no actual source span.

### 3.4 Failure modes Layer A must explicitly handle

- **Paraphrase drift.** The extractor produces a claim that says approximately what the source says but inverts a quantifier or modality. Caught by the NLI faithfulness check.
- **Speaker confusion.** Diarization mis-assigns a claim to a speaker who did not utter it. The platform should treat speaker attribution as a separate confidence score from the claim itself.
- **Citation collapse.** The extractor anchors many claims to the same large span (whole-paragraph laziness). The anchor module rejects citations larger than a configured maximum width.
- **Hallucinated claim.** The extractor invents a claim with no source basis. Caught by the citation-anchor verification pass.
- **Prompt injection at extraction.** A document instructs the extractor to behave maliciously. The LLM extractor must run in a constrained mode (system prompt strictly above user content, no tool use, JSON-only output) and the ingestion guard pre-filters the obvious patterns.

### 3.5 Reference implementation

| Component | Recommendation | Why |
|---|---|---|
| Live-capture client | PyQt6 (desktop) + a thin browser fallback using `MediaRecorder` | Native client gives reliable audio + filesystem; browser fallback for non-installable contexts. |
| ASR (local) | faster-whisper, model `small.en` or `medium` default | CTranslate2 backend; runs on CPU at 4–6x realtime, ∼200MB model. |
| ASR (cloud fallback) | OpenAI Whisper API | Quality benchmark and burst capacity. |
| Diarization | pyannote-audio 3.x | SOTA open-weights diarization; supports speaker enrollment. |
| PDF parsing | pypdf for digital; ocrmypdf (gates on env flag) for scanned | Pure-Python pypdf has no system deps; OCR only when needed. |

| Component | Recommendation | Why |
|---|---|---|
| LLM extractor | Claude or GPT-4-class, behind a router | Router lets you swap providers without touching extractor logic. |
| Linguistic fallback | spaCy `en_core_web_lg` + custom rule layer | Reliable, deterministic, available offline. |
| NLI faithfulness check | `cross-encoder/nli-deberta-v3-large` | SOTA accuracy on small NLI tasks, runs on CPU. |
| Ingestion guard | Custom regex + denylist + per-tenant rate limits | Must be small, auditable, non-LLM. |
| Job queue | Celery + Redis, or RQ for small deployments | Mature, well-instrumented, supports retries and dead-letter queues. |
| Object storage | S3-compatible (AWS S3 / R2 / MinIO) | Immutable artifacts; lifecycle policy for retention. |

## 4. Layer B — The Reasoning Engine

### 4.1 Purpose and boundary

Layer B is the brain. It receives validated `Claim`s with anchored citations from Layer A and is responsible for everything between that arrival and the moment a conclusion is ready to be surfaced in Layer C. Its outputs are `Principle`s (clustered, weighted claims), `Relationship`s (typed edges), `Conclusion`s (synthesized argued positions), `Forecast`s (probabilistic claims about future-resolvable events), and the rich metadata around all of them: coherence scores, peer-review notes, calibration history, and method invocation records.

Layer B is large. It is the layer in which the bulk of an organization's intellectual capital is encoded, and it is the layer that distinguishes an ICP from a content management system. The sub-sections that follow walk it from data model outward.

### 4.2 The data model

Listing 2: Core data model. These are the types that the entire engine traffics in.

```
record Principle {
   id: PrincipleId
   canonical_statement: String // the most refined wording
   cluster_member_claims: List<ClaimId>
   conviction: Float ∈[0,1] // weighted by frequency, emphasis, authority
   coherence_vector: CoherenceVector // see §4.4
   first_observed_at: Timestamp
   last_observed_at: Timestamp
   drift_history: List<DriftEvent>
   embedding: Vector
   tags: Set<String>

   invariant: cluster_member_claims is non-empty
   invariant: conviction is recomputed on every cluster modification
}

record Relationship { // typed edge between two claims, principles, or conclusions
   id: RelationshipId
   source_node: NodeId
   target_node: NodeId
   relation_type: enum {
      supports, contradicts, refines, instantiates,
      depends_on, generalizes, presupposes, conflicts_with
   }
   strength: Float ∈[0,1]
   detected_by: MethodId // which method asserted this edge
   detected_at: Timestamp
   evidence_claims: List<ClaimId> // the claims that justify this edge
}

record Conclusion { // a synthesized argued position, the publication unit
   id: ConclusionId
   title: String
   body: String // the argued case
   author: UserId
   descended_from: List<NodeId> // claims and principles the conclusion uses
   coherence_vector: CoherenceVector
   peer_review_state: PeerReviewState
```

```
    rigor_gate_state: enum { draft, in_review, blocked, published, retracted }
    revisions: List<RevisionId>
    related_conclusions: List<ConclusionId>
    open_questions: List<QuestionId>
}

record Forecast { // a future-resolvable prediction
    id: ForecastId
    statement: String // unambiguously resolvable
    resolution_criterion: String // exactly how it will be judged
    resolution_date: Date
    probability: Float ∈[0,1]
    method: MethodId
    issued_by: UserId
    issued_at: Timestamp
    descended_from: List<NodeId>
    resolution: enum { unresolved, true_, false_, ambiguous }
    resolved_at: Timestamp?
}

record Method { // a first-class persisted algorithm
    id: MethodId
    name: String
    version: SemVer
    interface: MethodInterface // declared input / output shapes
    implementation_ref: URI // pointer to the versioned code
    track_record: TrackRecord // see §4.13
    drift_baseline: Hash // canonical output on a frozen test set
    owner: UserId
}

record MethodInvocation { // every call to every method is recorded
    id: InvocationId
    method_id: MethodId
    method_version: SemVer
    input_hash: Hash
    output_hash: Hash
    started_at: Timestamp
    duration_ms: Int
    tenant_id: TenantId
    status: enum { succeeded, failed, refused }
    error: String?
}
```

**RATIONALE**

The `Method` and `MethodInvocation` types operationalize Invariant 7. Every transformation in the engine flows through a registered method; every call is recorded with input and output hashes; every method has a drift baseline against which silent regressions can be detected. The bookkeeping cost is modest — one row per invocation — and it is what makes the engine auditable.

### 4.3 The storage layer

The storage layer has three faces, intentionally separated so each can be optimized and scaled independently.

1. **The typed graph.** Nodes are `Claim`s, `Principle`s, and `Conclusion`s; edges are `Relationship`s. The graph is the canonical source of truth for the structural knowledge of the firm. Common queries: neighborhood retrieval, shortest-path between two claims, contradiction enumeration, cascade traversal (Section 4.7).
2. **The vector index.** A secondary structure mapping every embeddable node to its embedding. Used for semantic retrieval, similarity-based clustering, and the embedding-geometry layer of the coherence engine. The vector index is rebuildable from the graph; it is not authoritative.
3. **The signed audit ledger.** A strictly append-only, hash-chained log of every state-changing operation across the entire platform (not just Layer B). The ledger is what makes the platform's history reconstructible. See Section 6.1.

Listing 3: Storage interface.

```
interface KnowledgeStore {
   // Graph operations
   function upsert_claim(claim: Claim) →ClaimId
   function upsert_principle(p: Principle) →PrincipleId
   function add_relationship(r: Relationship) →RelationshipId
   function neighbors(node: NodeId, types: Set<RelationType>) →List<Edge>
   function contradictions_of(node: NodeId) →List<Edge>
   function cascade(conclusion: ConclusionId) →CascadeGraph // §4.7

   // Vector operations
   function similar_to(node: NodeId, k: Int) →List<(NodeId, Float)>
   function nearest(embedding: Vector, k: Int) →List<(NodeId, Float)>

   // Ledger operations
   function record_event(event: LedgerEvent) →EventId // append-only

   // Tenancy
   invariant: every read and write threads a TenantContext
   invariant: cross-tenant reads return empty without raising
}
```

### 4.4 The method registry

Every algorithm in the engine is registered with the `MethodRegistry` at startup. A registered method declares its interface (input type, output type, declared invariants), its current version, and the location of its implementation. Calls to a method go through the registry, which (a) records the invocation, (b) runs pre/post/failure hooks, (c) emits a ledger event, and (d) updates the method's track record.

Listing 4: Method registry pattern.

```
interface MethodRegistry {
   function register(method: Method) →()
   function invoke<I,O>(method_id: MethodId, input: I, ctx: TenantContext) →O
   function register_pre_hook(method_id: MethodId, hook: PreHook) →()
   function register_post_hook(method_id: MethodId, hook: PostHook) →()
   function register_failure_hook(method_id: MethodId, hook: FailureHook) →()

   invariant: every invoke() call records a MethodInvocation
   invariant: every invoke() call increments the method's track record counters
}
```

```
type PreHook = (Method, MethodInvocation, Input) →()
type PostHook = (Method, MethodInvocation, Input, Output) →()
type FailureHook = (Method, MethodInvocation, Input, Exception) →()
```

**RATIONALE**

The hook pattern is what lets cross-cutting concerns — ledger writes, observability spans, calibration tracking, drift detection — attach to every method invocation without method authors having to think about them. Method authors write the math; the registry handles the bookkeeping.

A new method ships in three steps: write the implementation against the declared interface, register it with a unique `MethodId`, ship a frozen test set whose output becomes the method's drift baseline. Thereafter, every invocation contributes to the method's track record, and a scheduled job (see Section 4.13) replays the test set against the current implementation and flags drift.

## 4.5 The coherence engine

The coherence engine is the heart of the platform. Given a claim, a principle, or a conclusion, it returns a *coherence vector* — not a scalar — with one component per validation layer. Aggregation into a single "coherence score" is a separate, configurable step done by an aggregator; the underlying components are always retained.

### *4.5.1 The composable layers*

A production ICP should ship with at least six independently-failing layers. They are listed in order of computational cost, ascending.

**Layer 1 — Logical consistency.**
An SMT-style check against an explicit list of formalized propositions in the immediate neighborhood. Uses Z3 or equivalent. Fast, decisive: returns a hard `inconsistent` verdict with a minimal contradicting set, or `satisfiable`. Catches direct logical contradictions and quantifier conflicts.

**Layer 2 — Natural-language inference.**
A cross-encoder NLI model scores entailment and contradiction between the target claim and each of its k nearest graph neighbors. Returns three probabilities (entail / neutral / contradict) and a list of contradicting neighbors.

**Layer 3 — Probabilistic consistency.**
If the claim has a numeric component (probability, frequency, magnitude), check Bayesian consistency against related quantitative claims. Catches the case where two analysts assert probabilities that cannot both be true under any prior.

**Layer 4 — Embedding geometry.**
Uses difference-vector analysis and learned contradiction directions in embedding space. Cosine similarity alone is insufficient (the "cosine paradox": contradictory statements often have high cosine similarity because they share subject matter). The contradiction direction is learned offline from a labeled contradiction dataset; at inference, the projection of the difference vector onto this direction is the contradiction signal.

**Layer 5 — Information-theoretic novelty.**
Measures the surprise of the claim against the existing graph. A claim that is highly predicted by existing claims contributes little; a claim that is poorly predicted is either novel insight or noise. Returns surprisal in bits.

**Layer 6 — LLM judgment.**
A frontier LLM evaluates the claim against retrieved neighbors with a structured rubric. This is the slowest layer and is treated as an opinion, not an oracle. Required to disagree explicitly with the lower layers when it does; agreement adds little new signal, disagreement is informative.

Listing 5: Coherence engine interface.

```
record CoherenceVector {
   logical: VerdictWithEvidence
   nli: EntailmentDistribution
   probabilistic: ProbabilisticVerdict
   geometric: Float ∈[-1, 1] // signed contradiction projection
   information: Float // surprisal in bits
   llm: StructuredVerdict
}

interface CoherenceEngine {
   function score(target: NodeId, k_neighbors: Int = 20) →CoherenceVector
   function aggregate(v: CoherenceVector) →AggregatedScore
   invariant: components of CoherenceVector are produced by independent methods
   invariant: aggregation function is itself a registered Method
}
```

**Composability invariant**

A new validation layer should be addable without modifying any existing layer or the aggregator's interface. The aggregator reads the vector by name, not by position; new layers extend the vector type.

### 4.6 The inference engine: forward and inverse

The inference engine answers two distinct questions, and the platform must support both.

**Forward inference.** "Given the firm's body of principles, what should we believe about $X$?" Retrieves relevant principles, weights them by conviction and topical relevance, generates a candidate analysis, runs it through the coherence engine, and returns an answer with its evidence chain.

**Inverse inference.** "Given that we observed outcome $Y$, what set of principles is most consistent with that outcome, and what did our prior body have to say about it?" This is the inference path used for post-mortems, calibration analysis, and revision of standing positions in light of new evidence.

Listing 6: Inference engine interface.

```
interface InferenceEngine {
   function forward(question: String, ctx: ReasoningContext) →ReasonedAnswer
```

```
    function inverse(observed_outcome: Outcome) →List<(PrincipleId, ConsistencyScore)
        >
    invariant: every ReasonedAnswer carries its evidence chain
    invariant: forward and inverse both emit MethodInvocations
}

record ReasonedAnswer {
    answer: String
    confidence: Float ∈[0,1]
    evidence_chain: List<(NodeId, RelevanceWeight, RoleInArgument)>
    counter_considerations: List<NodeId> // what would falsify this
    open_questions: List<QuestionId>
    method_invocations: List<InvocationId> // for full reproducibility
}
```

The `counter_considerations` field is non-optional and is what distinguishes an answer from a confident-sounding paragraph. Every forward inference must enumerate the conditions under which its answer would be wrong, drawn from actual contradicting claims in the graph.

### 4.7 Cascade — provenance traversal

The cascade module exposes the provenance graph as a first-class queryable structure. Given a `Conclusion`, it returns the directed acyclic graph of all the `Principle`s and `Claim`s the conclusion descends from, annotated with how load-bearing each node is.

Listing 7: Cascade interface.

```
record CascadeGraph {
    root: ConclusionId
    nodes: List<NodeWithLoad>
    edges: List<DescentEdge>

    invariant: graph is a DAG
    invariant: every leaf is a Claim (terminates at a primary source)
}

record NodeWithLoad {
    node: NodeId
    load_bearing: Float ∈[0,1] // removing this node degrades the conclusion by this
        much
    distance_from_root: Int
}

interface Cascade {
    function trace(conclusion: ConclusionId) →CascadeGraph
    function diagnose(conclusion: ConclusionId) →CascadeDiagnostic
    // diagnostic: identifies single points of failure, weak links, citation gaps
    function render(g: CascadeGraph, format: enum {svg, dot, json}) →Bytes
}
```

Cascade diagnostics surface three failure modes adopters care about: a *thin foundation* (a conclusion descended from only one or two source claims), an *ancestor contradicted later* (a claim the conclusion relies on has since been contradicted by a higher-conviction claim), and a *stale ancestor* (a load-bearing claim that has aged past its decay threshold; see Section 4.10).

### 4.8 Peer review — multi-perspective evaluation

A conclusion that has been scored by the coherence engine has been checked for internal consistency. That is necessary but not sufficient. The peer-review module submits each conclusion to a panel of *reviewer methods*, each of which evaluates the conclusion from a single explicit perspective.

The reference panel has seven reviewers; an organization may add or replace reviewers but should not remove all of them.

**Methodological.**
Was the right method used for this kind of claim? Is the method's track record adequate in this domain?

**Evidential.**
Does the evidence chain actually entail the conclusion, or is there a leap? Are the cited sources primary or derivative?

**Statistical.**
If the conclusion makes quantitative claims, are the statistics sound? Sample sizes, base rates, confidence intervals.

**Adversarial literature.**
Does the external literature have published critiques of the methods or claims used? Searches a curated corpus of methodological criticism.

**Replication.**
Has anyone else — inside or outside the firm — arrived at the same conclusion using independent methods or data?

**Rhetorical.**
Is the argument's confidence proportional to its evidence? Does the language overclaim? Are caveats present where they should be?

**Humility.**
Does the conclusion explicitly state what would falsify it? Does it acknowledge uncertainty in the right places?

Listing 8: Peer review interface.

```
interface Reviewer {
   function review(c: Conclusion, ctx: ReviewContext) →ReviewerVerdict
   function perspective() →String
}

record ReviewerVerdict {
   perspective: String
   verdict: enum { accept, accept_with_revisions, reject, abstain }
   notes: String
   flagged_passages: List<Span>
   suggested_revisions: List<RevisionSuggestion>
}

interface PeerReviewPanel {
   function convene(c: Conclusion, reviewers: List<Reviewer>) →PanelOutcome
   invariant: reviewer outputs are stored separately; no consensus is computed
       automatically
```

```
   invariant: a conclusion may not pass the rigor gate (§4.11) without a complete
       panel outcome
}
```

**RATIONALE**

Reviewer outputs are stored separately and a consensus is *not* computed automatically. The temptation to reduce seven independent perspectives to a single number is what destroyed the academic peer-review process. The platform's job is to surface the seven verdicts to a human editor who decides; the platform's job is not to be the editor.

### 4.9 Adversarial generation — counter-claims by construction

For every published conclusion, the adversarial module generates the strongest counter-claims it can find or construct. Three sources feed it: existing contradicting claims in the graph; published critiques retrieved from the adversarial-literature corpus; and LLM-generated counter-arguments produced under an explicit "steelman the opposite" prompt. Each counter-claim is itself ingested as a first-class `Claim` with the conclusion as its target. A conclusion that survives the adversarial pass is annotated with the counter-claims it withstood; a conclusion that does not is sent back to the author with the strongest counter as the suggested revision.

### 4.10 External battery — sanity-check against the outside world

The external battery is a module that periodically tests the platform's claims and methods against external benchmarks. The reference battery includes:

- **Prediction markets** (Metaculus, Manifold, Polymarket): for each `Forecast` the platform issued, compare against the market's contemporaneous price.
- **Good Judgment Project** or equivalent: replay GJP-style questions through the platform's forecast method and compare the platform's Brier score against the established human benchmark.
- **ClaimReview corpus**: for any factual claim the platform endorsed, check against the fact-checking corpus (Snopes, PolitiFact, etc. as structured ClaimReview JSON-LD).
- **Replication datasets**: for empirical claims drawn from published research, check against the Open Science Framework's replication outcomes.

Listing 9: External battery interface.

```
interface ExternalAdapter {
   function fetch_relevant(node: NodeId) →List<ExternalRecord>
   function compare(node: NodeId, record: ExternalRecord) →ComparisonResult
}

interface ExternalBattery {
   function register_adapter(name: String, adapter: ExternalAdapter) →()
   function run(scope: BatteryScope) →BatteryReport
   invariant: failures are surfaced to the rigor gate; passes are silent
}
```

### 4.11 The rigor gate — the publication checkpoint

The rigor gate is the single chokepoint through which all candidate publications must pass. Its job is to refuse to publish output that fails one of a list of registered, hard-edged checks. A check is a small, deterministic function with three outcomes: `pass`, `block`, or `require_override`. Overrides require a named human and a written justification; both are written to the ledger.

The reference gate ships with at least the following checks:

- **Provenance complete.** Every load-bearing claim has a valid `CitationSpan`.
- **Coherence threshold.** No component of the conclusion's coherence vector is below a configured minimum.
- **Peer-review complete.** A panel outcome exists with no `reject` verdicts unaddressed.
- **Adversarial survived.** The strongest generated counter-claim has been rebutted or acknowledged.
- **Unresolved honesty.** If the conclusion makes claims in the presence of unresolved open questions, those questions are explicitly listed.
- **No stale ancestors.** No load-bearing ancestor is past its decay threshold.
- **Calibration sanity.** If the author has a track record, the confidence asserted is consistent with their historical calibration in this domain.

Listing 10: Rigor gate interface.

```
interface RigorGate {
   function evaluate(c: Conclusion) →GateOutcome
   function register_check(check: GateCheck) →()
   function override(check: GateCheck, justification: String, by: UserId) →
       OverrideRecord
}

record GateOutcome {
   decision: enum { pass, block, require_override }
   failed_checks: List<CheckFailure>
   overrides_applied: List<OverrideRecord>
}
```

**Refusal is a feature**

The rigor gate exists to be obstructive. A platform that publishes everything is a platform that has not built one. The refusal dashboard (Section 5.2) makes refusals visible so they can be inspected, contested, and used to refine the checks.

### 4.12 Mitigations — the standing defenses

Mitigations are a small collection of always-on guards that protect the engine from specific known failure modes. They are different from the rigor gate: gates are one-shot evaluations at publication time; mitigations run continuously.

**Ingestion guard.**
Already covered in Layer A. Pre-extraction filter.

**Citation anchor.**
Already covered in Layer A. Verifies anchored spans entail the claim.

**Temporal guard.**
Prevents the use of claims whose source post-dates the time horizon of a forecast or analysis (data-leakage prevention for retrospective evaluation).

**Embedding text safety.**
Strips or escapes special characters before they reach the embedding model, which can otherwise produce degenerate embeddings.

**Tenant audit.**
Records every cross-tenant boundary read attempt. Cross-tenant reads always return empty; the attempt itself is the audit signal.

**Calibration guard.**
Refuses to issue a forecast whose stated confidence exceeds the issuing method's historical calibration in the domain.

### 4.13 Evaluation & decay

The evaluation module maintains the platform's understanding of how well its methods are working.

**Slicer.**
Partitions outcomes by method, by topic, by author, by time window. Without slicing, calibration metrics are meaningless aggregates.

**Outcomes.**
Records the resolution of every `Forecast` and every quantitative claim. The outcome layer is what makes the track record possible.

**Counterfactual.**
For each major decision the platform recommended, replays the decision with one input removed (a principle, a source, a method) and measures the recommendation's stability.

**Calibration metrics.**
Brier score, log loss, reliability diagrams, all sliced. A method's track record is a structured record of these over time.

The **decay** module is the active counterpart. It monitors the freshness of every node in the graph and applies configured retirement policies. A principle whose underlying claims are all more than $N$ months old, and which has not been reinforced by any recent claim, decays from `active` to `archival`. Archival nodes are still queryable but are excluded from the conviction-weighted retrieval that feeds forward inference.

Listing 11: Decay interface.

```
record DecayPolicy {
   domain: String // e.g. "macroeconomic", "first principles"
   half_life_months: Float
   floor_conviction: Float // conviction below which a principle is retired
   reinforcement_extension: Bool // does a new supporting claim reset the clock?
}
```

```
interface Decay {
   function register_policy(p: DecayPolicy) →()
   function tick() →List<RetirementEvent> // run on a schedule
   invariant: retirement events are appended to the ledger; nothing is deleted
}
```

### 4.14 Synthesis & transfer

**Synthesis** produces conclusions and reports from the graph: given a topic and a scope, it retrieves the relevant principles and claims, generates a draft conclusion, runs it through coherence + peer review + adversarial + rigor gate, and emits a candidate publication. Synthesis is the operation that turns the graph back into prose that humans can read.

**Transfer** packages methods (and optionally subsets of the graph) for export. An organization that wants to share or sell its methodology needs to be able to (a) freeze a versioned method bundle, (b) sign it, (c) provide a Docker image that runs the bundle against a declared input contract, and (d) ship a test harness that lets the recipient verify the bundle behaves as advertised on the published test set. The transfer module exists because methods are the product (Invariant 7) and a product must be shippable.

Listing 12: Transfer interface.

```
interface Transfer {
   function freeze_method(m: MethodId, version: SemVer) →FrozenMethodBundle
   function sign(bundle: FrozenMethodBundle, key: SigningKey) →SignedBundle
   function build_docker(bundle: SignedBundle) →DockerImageRef
   function build_harness(bundle: SignedBundle, test_set: TestSet) →HarnessBundle
   invariant: a signed bundle is immutable; new versions ship as new bundles
}
```

### 4.15 Reference implementation

| Component | Recommendation | Why |
|---|---|---|
| Language | Python 3.11+ with Pydantic v2 for data models | Mature ML/NLP ecosystem; Pydantic gives runtime contract enforcement. |
| Graph store | Postgres with adjacency tables + JSONB, or Neo4j if scale demands | Postgres scales further than people think; Neo4j only when query patterns are deeply graph-shaped. |
| Vector index | pgvector co-located in Postgres, or Qdrant if isolation needed | Co-location avoids two-store consistency problems. |
| Embeddings | SBERT `all-mpnet-base-v2` baseline; BGE-large-en-v1.5 for quality | SBERT is well-supported; BGE is a stronger replacement when GPU is available. |
| Logical layer (L1) | Z3 SMT solver via z3-py | Industry-standard; mature Python bindings. |

| Component | Recommendation | Why |
|---|---|---|
| NLI (L2) | DeBERTa-v3 cross-encoder | SOTA on small NLI; runs on CPU at acceptable latency. |
| Probabilistic (L3) | Custom Bayesian consistency on PyMC or a hand-rolled checker | Inputs are sparse; full PyMC is overkill for most checks. |
| Geometry (L4) | numpy / scikit-learn for difference-vector analysis; learned contradiction direction | The contradiction direction is the most empirically validated novel signal. |
| Information (L5) | Custom surprisal computation using a small LM for likelihoods | A small LM is sufficient; the signal is order-of-magnitude novelty. |
| LLM judge (L6) | Frontier LLM via the same router used in Layer A | Treat the LLM as opinion, not oracle. |
| Method registry | Custom registry with pre/post/failure hooks (the Theseus pattern is sound) | The hook system is what makes cross-cutting concerns tractable. |
| Schema migrations | Alembic | Mature, declarative, version-controlled. |
| Job queue | Celery or Temporal | Celery is simpler; Temporal is better for long-running workflows. |
| Observability | OpenTelemetry + structlog + Grafana | Method invocations are spans; ledger events are logs; calibration is a metric. |
| Signing keys | Ed25519 via cryptography library | Fast, small, modern. |

## 5. Layer C — Repository & Surfaces

### 5.1 Purpose and boundary

Layer C is the human-facing skin: the web application (or applications) through which members of the organization read, write, query, dispute, and publish using the engine below. Layer C does not contain reasoning logic — every non-trivial operation is a call into Layer B. Its job is presentation, navigation, and the human workflows around the engine: authoring conclusions, queuing reviews, browsing the methods registry, contesting a rigor-gate refusal, watching a calibration chart, publishing to an external audience.

### 5.2 The user-facing data model

Layer C has its own small data model, sitting alongside the engine's. It owns users, sessions, attributions, attachments, and the audit of human actions. It does not own claims, principles, or conclusions — those belong to the engine.

Listing 13: Repository data model.

```
record User {
   id: UserId
   auth_subject_id: String // identifier from auth provider
   handle: String // unique
   display_name: String
   email: String
   role: enum { reader, contributor, editor, operator, admin }
   bio: String?
   avatar_uri: URI?
   created_at: Timestamp
}

record Entry { // the user-facing wrapper for any contribution
   id: EntryId
   title: String
   kind: enum { note, article, transcript, voice, art, discussion, response }
   discipline: String // org-defined taxonomy
   body: String?
   media_path: URI?
   transcript: String?
   summary: String?
   tags: Set<String>
   created_at: Timestamp
   updated_at: Timestamp

   // Each entry can be linked to nodes in the engine
   linked_claims: List<ClaimId>
   linked_principles: List<PrincipleId>
   linked_conclusions: List<ConclusionId>
}

record Attribution {
   id: AttributionId
   user_id: UserId
   entry_id: EntryId
   role: enum { author, co_author, editor, reviewer }
   created_at: Timestamp
}
```

```
record Attachment {
   id: AttachmentId
   entry_id: EntryId
   user_id: UserId
   kind: enum { link, file }
   uri: URI?
   file_path: URI?
   file_name: String?
   mime_type: String?
   label: String?
}

record AuditEvent { // Layer C's view of the ledger
   id: AuditEventId
   user_id: UserId
   entry_id: EntryId?
   action: String
   detail: String?
   created_at: Timestamp
   ledger_event_id: EventId // pointer to the canonical ledger
}
```

### 5.3 Surface inventory

A production ICP has surprisingly many distinct UI surfaces. The list below is the minimum reasonable set; an organization should not ship without coverage of every category, though it may implement many of them as bare functional pages before investing in polish.

#### *5.3.1 Authentication and account*

- **Login, signup, password change.** Provider-backed (Supabase Auth, NextAuth, or equivalent). TOTP 2FA non-optional for editors and above.
- **Account profile.** Handle, display name, avatar, bio.
- **Admin / contact / org management.** Role assignment, tenant settings.

#### *5.3.2 Capture-facing surfaces*

- **Contribute.** The general entry-creation form: type, title, body, attachments. The single surface that produces user-authored Layer A inputs.
- **Reading queue.** Curated inbox of articles, papers, and external claims pending triage.
- **Source triage.** Decide whether a queued source goes into ingestion, the literature corpus, or is rejected.
- **Transcripts.** View per-source diarized transcripts; navigate by speaker; jump to anchored claims. Includes a per-transcript "conversation geometry" visualization that surfaces argumentative structure.
- **Sessions / reflection.** Post-session view for a captured conversation, showing extracted claims, contradictions detected during the session, and a reflection log.

#### *5.3.3 Reasoning-engine-facing surfaces*

- **Conclusions.** The primary publication unit. A conclusion has tabs for body, cascade (provenance), peer-review verdicts, adversarial counter-claims, related conclusions, version history, blindspots panel (what the engine thinks the conclusion is not addressing), and a revision preview. A canonical sigil / fingerprint is computed per conclusion and rendered for visual identity in citations.
- **Cascade view.** Standalone provenance visualization for any conclusion. Interactive: click any ancestor to see its own neighborhood.
- **Contradictions.** A browseable list of detected contradictions across the graph, sorted by load-bearing weight.
- **Open questions.** The accumulated list of unresolved questions tied to conclusions and principles.
- **Post-mortem.** Per-conclusion (or per-forecast) post-mortem: what we predicted, what happened, what the diff implies for our methods.
- **Explorer.** Free exploration of the graph: scatter plots of embedding-space projections, neighborhood graphs, principle clusters.
- **Founders / voices.** If the organization captures named individuals' bodies of work, a per-individual view of their principles, conviction, drift, and characteristic arguments.
- **Literature.** The external-literature corpus indexed by the engine.
- **Papers.** The internal-and-external paper queue.

#### *5.3.4 Methods and meta-methodology surfaces*

- **Methods registry.** Browse every registered method: interface, version, track record, drift history. Per-method pages with failure analysis, domain coverage, replication guide, and known blindspots.
- **Methods graph.** A view of methods as nodes and their composition relationships as edges.
- **Method drift panel.** For a given method, the drift signal over time against its baseline test set.
- **Methodology pages.** Per-method documentation: how to replicate, where it has failed, what its domain is, what its track record is, what its known blindspots are.

#### *5.3.5 Rigor and adversarial surfaces*

- **Rigor-gate queue.** The list of conclusions currently blocked at the gate, the failing checks, and the override interface (with required justification, written to the ledger).
- **Refusal dashboard.** Monthly statistics on refusals — which checks fire most, with what overrides — so the gate can be tuned.
- **Peer-review queue.** The review backlog, the panel of seven verdicts per conclusion, and the editor's decision interface.
- **Adversarial dashboard.** For each published conclusion, the strongest generated counter-claims and the rebuttals.
- **Q-review.** A spot-check queue for random samples of any registered method's recent outputs.

#### *5.3.6 Forecast and calibration surfaces*

- **Forecasts list.** All issued forecasts with their probabilities, resolution criteria, status.
- **Per-forecast page.** Title, statement, resolution criterion, probability over time, source drawer (the cascade), audit trail, chat panel, resolution panel.
- **Portfolio.** Calibration chart, distribution histogram, paper-PnL chart, Brier-score over time, status strip, bet log.
- **Operator console.** The kill-switch and bet-stream view for the operator who supervises automated forecast issuance.

#### *5.3.7 Currents and live-news surfaces*

- **Currents.** The live ingest of external developments (news, social, market signals) the engine is monitoring. Topic clusters, source drawer, audit trail, follow-up chat.
- **Founder-currents.** Per-individual recent activity within the captured corpus.

#### *5.3.8 Evaluation and operations surfaces*

- **Eval.** Run history of evaluation passes; per-run metrics; counterfactual reports.
- **Decay.** The retirement queue: nodes scheduled to decay, nodes that have decayed, the configured policies.
- **Scoreboard.** The firm's calibration over time, sliced by method, author, topic.
- **Provenance.** Org-wide provenance search: "find every conclusion that descends from this source".

#### *5.3.9 Public-facing surfaces*

- **Publication.** The interface authors use to push a passed conclusion to the public site.
- **Post / public conclusion.** The rendered, externally-readable conclusion, with a reader-response form and a visible sigil.
- **Reader responses.** The inbox of incoming reader objections, themselves first-class claims subject to the engine's checks.
- **Social.** Cross-posting and engagement surface for published conclusions.
- **Revisions.** The diff between conclusion versions, with a justification trail.

### 5.4 Reference implementation

| Component | Recommendation | Why |
|---|---|---|
| Framework | Next.js 15 (App Router) + TypeScript | SSR for SEO of public conclusions; server actions for typed RPC. |

| Component | Recommendation | Why |
|---|---|---|
| ORM | Prisma against Postgres | Schema-as-code; the engine and the repository share the same Postgres. |
| Auth | Supabase Auth or NextAuth with TOTP 2FA | 2FA non-optional above contributor; SSO via OIDC for org deployments. |
| Realtime | Supabase Realtime, Pusher, or SSE | Currents and forecast streams need push updates. |
| Charts | Recharts, Visx, or Plotly | Calibration / Brier / portfolio views are chart-heavy. |
| Graph viz | D3 + Cytoscape.js (or react-flow) | Cascade view needs interactive DAG layout. |
| Markdown / writing | MDX or TipTap | Conclusions are long-form; need footnotes, callouts, citations. |
| File storage | S3-compatible bucket (R2 / S3 / Supabase) | Same bucket as Layer A for the artifacts. |
| Background work | Inngest, Trigger.dev, or hand-rolled Celery bridge | UI triggers long-running engine jobs without blocking. |
| Search | Postgres full-text + pgvector + a thin BM25 layer | Two-stage retrieval: BM25 candidates re-ranked by vector. |
| Deployment | Vercel or self-hosted Next + dedicated Postgres | Vercel is fastest path; self-host when data residency matters. |

## 6. Cross-cutting concerns

### 6.1 The signed audit ledger

The ledger is the single most important piece of infrastructure in the platform, and the easiest to underbuild. It is a strictly append-only, hash-chained log of every state-changing operation across all three layers. Every `Source` registered, every `Claim` extracted, every `Relationship` added, every `Method` invoked, every `Conclusion` published, every rigor-gate decision, every override, every retirement, every cross-tenant attempt — all of it lands in the ledger.

Listing 14: Ledger interface.

```
record LedgerEvent {
   id: EventId
   timestamp: Timestamp
   tenant_id: TenantId
   actor: ActorRef // user, method, or system
   action: String // canonical verb, e.g. "claim.extract"
   subject: NodeRef // what was acted upon
   payload_hash: Hash // SHA-256 of the canonical payload JSON
   prev_event_hash: Hash // hash of the previous event in this tenant
   this_event_hash: Hash // hash(timestamp || action || subject || payload_hash ||
       prev_event_hash)
   signature: Bytes // Ed25519 signature over this_event_hash by the ledger key

   invariant: events are immutable once written
   invariant: prev_event_hash chain is unbroken per tenant
}

interface Ledger {
   function append(event: LedgerEvent) →EventId
   function range(tenant: TenantId, from: Timestamp, to: Timestamp) →Stream<
       LedgerEvent>
   function verify(tenant: TenantId) →VerificationReport
   function export(tenant: TenantId, format: enum {jsonl, parquet}) →URI
}
```

The ledger has three downstream consumers: the audit-event view that Layer C surfaces to users, the verification job that periodically re-validates the hash chain, and the backup/export pipeline that ships ledger snapshots to cold storage. The ledger is what allows the platform to answer the question "what changed and who changed it" without ambiguity, even years later.

**Treating the ledger as logging**

A standard log line in stdout or a JSON file is not a ledger. Logs are mutable, unhashed, and frequently rotated. A ledger must be hash-chained and signed, with verification jobs that fail loudly when the chain is broken. The temptation to "just write to the application log and call it audit" is the most common reason adopters cannot answer the provenance question (Section 2.3) two years in.

### 6.2 Tenancy

If the platform will ever serve more than one organization, tenancy must be designed in from the start. Adding tenancy to a single-tenant system later is a substantial rewrite. The minimum design:

Listing 15: Tenant context.

```
record TenantContext {
   organization_id: OrgId
   slug: String
}

interface Tenanted {
   invariant: every public function reads a TenantContext
   invariant: every store query filters by ctx.organization_id at the data layer
   invariant: cross-tenant reads return empty without raising
   invariant: cross-tenant attempts are written to the tenant_audit mitigation
}
```

Three implementation details that matter:

- The `organization_id` column is on every persisted table that contains tenant data — yes, every one. Joining across tables without preserving the filter is the most common cross-tenant leak.
- Even single-tenant deployments thread `TenantContext.system_default()`. This means moving from laptop to multi-tenant is a configuration change, not a code change.
- The `tenant_audit` mitigation (Section 4.12) is what makes leaks visible. The leak attempt returns empty data — this is what protects users — but the attempt itself is the security signal.

### 6.3 Observability

Three observability pillars, mapped to the platform's three first-class objects:

**Traces.**
One trace per ingestion run and one trace per inference call. Spans inside the trace correspond to method invocations. The trace ID is recorded on every `MethodInvocation` and every `LedgerEvent` so a single user-visible action can be reconstructed end-to-end.

**Metrics.**
Method-level: invocations / failures / p50/p95/p99 latency / drift signal. Engine-level: graph size, vector index size, ledger event rate, queue depth. Business-level: forecasts issued, conclusions published, refusals, calibration over rolling windows.

**Logs.**
Structured, key-value, with the tenant ID and trace ID on every line. Stack traces and warnings only; routine state changes go to the ledger, not logs.

OpenTelemetry plus a managed backend (Honeycomb, Datadog, Grafana Cloud) is the cheapest path. Self-hosting the observability stack is feasible but is a meaningful operational commitment.

### 6.4 Configuration and environment validation

The platform should refuse to start with an invalid configuration. The minimum validation step verifies that every required environment variable is set, every external dependency is reachable, every signing key is loadable, and every registered method's drift baseline is present. Failures at startup are recoverable; failures at the first real request are not.

The configuration system should be hierarchical (org-default → tenant-override → environment-override) and every value's effective source should be inspectable from an admin view — "where did this threshold come from?" is a question that gets asked at every postmortem.

### 6.5 Schema migrations

Both the engine and the repository schemas evolve. Migrations are managed with a declarative tool (Alembic for Python; Prisma Migrate for TypeScript) and run as part of deployment. Two rules:

- Migrations are forward-only. Backward migrations are written for the explicit purpose of recovery, not regular use.
- Migrations that drop columns or change column types ship as two deployments: the first deploys reads-and-writes-both, the second deploys reads-and-writes-the-new-only. This applies even when the team thinks no downstream depends on the old shape.

### 6.6 Backup and restore

The backup target is not a database snapshot — it is the full state of the platform such that a fresh deployment of the same software version can be brought up against the restored state and produce identical engine outputs on identical inputs. The minimum backup set:

- Postgres dump (graph, vector, repository data, ledger).
- Object storage mirror (source artifacts, attachments).
- Method drift baselines (the canonical test outputs that current methods are checked against).
- Signing keys (escrowed, separately).
- Configuration snapshot.

Restore is tested quarterly. A backup that has never been restored is not a backup.

## 7. Build phases and milestones

The full platform is at least an 18–24 month build for a small team. Adopters who try to ship it all at once invariably ship none of it. The phasing below is the recommended slicing.

### 7.1 Phase 0 — Skeleton (weeks 1–4)

- Postgres + pgvector + Alembic + Prisma + Next.js scaffold.
- Auth (Supabase) with two roles: contributor and editor.
- `Source`, `Claim`, `Principle`, `Relationship` schema.
- Ledger with hash chaining; verification job.
- Tenancy threaded through every store call from day one.
- A single seed source (text upload) end-to-end into a single claim end-to-end into a single graph view.

**Exit criterion:** a contributor can upload a PDF, the system extracts one claim with a citation span, and the claim is visible in a list view with its source.

### 7.2 Phase 1 — Ingestion (weeks 5–10)

- Audio capture (browser `MediaRecorder`) + ASR (faster-whisper) + diarization (pyannote).
- LLM extractor with strict JSON schema + linguistic fallback.
- NLI faithfulness check.
- Citation anchor with verification pass.
- Ingestion guard with at least three pattern checks.
- Source registry with content fingerprinting.

**Exit criterion:** a captured audio session produces diarized, anchored, faithfulness-checked claims with no manual intervention.

### 7.3 Phase 2 — Coherence and graph (weeks 11–18)

- Method registry with hooks.
- Coherence layers L1 (logical, Z3) and L2 (NLI) and L4 (geometric, with a small contradiction dataset).
- Principle clustering with conviction weighting.
- Forward inference with evidence chain.
- Cascade traversal and visualization.
- Contradiction enumeration.

**Exit criterion:** a user can ask a question, get a reasoned answer with an evidence chain, see the cascade behind it, and surface the most contradictory claims in the graph.

### 7.4 Phase 3 — Rigor (weeks 19–26)

- Three peer-review reviewers (methodological, evidential, humility).
- Adversarial counter-claim generator.
- Rigor gate with three checks (provenance complete, coherence threshold, peer-review complete).
- Refusal dashboard.
- Forecast model with resolution tracking and calibration metrics.

**Exit criterion:** a conclusion cannot be published without passing the gate; the refusal dashboard shows what fails and why.

### 7.5 Phase 4 — External grounding (weeks 27–36)

- External battery with at least two adapters (Metaculus and a fact-checking corpus).
- Coherence layers L3, L5, L6.
- Remaining four peer-review reviewers.
- Decay module with at least one policy per domain.
- Inverse inference and post-mortem workflow.
- Method drift monitor.

**Exit criterion:** the platform's calibration is being tracked against an external benchmark; methods are checked for drift on a schedule.

### 7.6 Phase 5 — Transfer and surfaces (weeks 37–52)

- Method bundle freezing, signing, docker harness.
- Currents and live ingestion.
- Public publication surface with reader-response intake.
- Operator console with kill-switch for any automated issuance.
- Backup / restore automation; restore drill.

**Exit criterion:** the methodology is shippable as a signed, dockerized bundle; the public publication path is live; the team has restored from a backup at least once.

## 8. Where this template is opinionated, and where it leaves room

The seven invariants in Section 1.3 are non-negotiable. Architectures that violate any of them are not Intellectual Capital Platforms in the sense this document means. Everything else is implementation choice; this section makes the distinction explicit so adopters know what they are signing up for and what they remain free to change.

### 8.1 Opinionated

- **Three layers, in this order.** Capture, Reasoning Engine, Repository. The reasoning engine must not call back up into Layer C, and Layer C must not contain reasoning logic.
- **Atomic-claim granularity.** Paragraph-level or document-level storage is not adequate. Coherence checks below paragraph-level fall apart for trivial reasons; storage above paragraph-level cannot support them.
- **Typed graph as primary store.** A vector-only store cannot support cascade or contradiction enumeration. A relational-only store cannot support neighborhood retrieval at scale. A typed graph with a vector secondary index is the design that supports both.
- **Signed, hash-chained ledger.** Plain logs are not a substitute. The ledger is what makes the platform's history reconstructible.
- **First-class methods with track records.** Treating methods as code rather than as data is what reduces a platform to a content tool.
- **The rigor gate.** A platform that publishes everything has not built a rigor gate.
- **Multi-perspective peer review.** A single reviewer is not enough; an aggregated consensus is worse than enough. Surface the panel; let the editor decide.

### 8.2 Open to choice

- Programming languages. Python is recommended for Layer B because of the ML ecosystem; the rest is free. The interfaces in this document are language-neutral.
- LLM provider. The architecture treats LLMs as routable services. Vendor change is a configuration change.
- Embedding model. Replaceable per domain; embedding choice is a per-tenant configuration.
- Whether to do live audio capture at all. Some organizations have abundant transcription already; the live-capture client is optional in those settings.
- Whether to deploy multi-tenant or single-tenant. The architecture supports both, with the same code.
- Web framework. The recommendation is Next.js because of its SSR support for public publication, but any modern framework with strong RPC and realtime support is workable.
- Storage substrate. Postgres + pgvector covers the vast majority of deployments; Neo4j or Qdrant become defensible at scales most adopters will not reach.

## 9. Anti-patterns and known failure modes

These are the patterns that have, in the experience of building real platforms of this kind, repeatedly destroyed adopter projects. They are listed here so that they may be recognized early.

**Conflating retrieval with reasoning**

"We have RAG, therefore we have an ICP." Retrieval-augmented generation is a useful technique inside Layer B's inference component; it is not, by itself, an Intellectual Capital Platform. Retrieval finds relevant text; reasoning evaluates it for consistency, validates its provenance, situates it in a graph, runs it through coherence checks, submits it to peer review, and gates its publication. A vector store with an LLM in front is the first ten percent of the platform.

**Storing summaries instead of claims**

"We will store the summary of each meeting and reason over the summaries." Summaries are lossy in the precise ways that matter: they elide modality, quantification, and the speaker-level disagreements that contradiction detection depends on. The unit must be the atomic claim. Summaries are derivatives; they belong in the surface layer, not the store.

**Letting the LLM be the oracle**

"We will trust the LLM to judge coherence." The LLM is one of six layers and is treated as an opinion, not an oracle. It is required to disagree explicitly with the lower layers when it does. A platform whose coherence score is a single LLM call is a platform that will silently inherit the model's biases as "the firm's view".

**Aggregating peer-review panels into consensus**

"We'll average the seven reviewers and call it a score." Averaging seven independent perspectives into one number destroys the only useful thing about having seven perspectives. Surface each verdict; let the human editor see the spread; let disagreement be visible. The reductionist temptation here is the same one that hollowed out academic peer review.

**Treating provenance as a UI feature**

"We'll show the sources beneath each conclusion in the UI." Provenance is not a UI feature; it is a storage and reasoning invariant. If it is implemented in Layer C as a list of links rendered next to the conclusion, it will rot the first time the engine's data shape changes. Provenance must live in the cascade structure inside the engine, queryable, traversable, diagnosable.

**Skipping the ledger because "we have logs"**

Covered above; restated here because it is the most common omission. A standard log file is not a ledger.

**Skipping tenancy because "we are single-tenant for now"**

Tenancy is a property of every database query, every store interface, every method invocation. Retrofitting it is a multi-quarter undertaking. Thread `TenantContext` through every interface on day one, default to `system_default()`, and the move to multi-tenant becomes a configuration change.

**A rigor gate that always passes**

A rigor gate that never refuses is theater. Track the refusal rate. If it has been zero for a month, the gate is mis-configured or unused. Refusals are the signal that the gate is working.

**Methods that are functions, not records**

"We just call `score_coherence(x)` directly." Methods called as functions cannot be tracked, hooked, drift-monitored, or shipped. The cost of routing every call through a registry is one wrapping layer; the value is everything that depends on methods being first-class objects.

**Privileging speed over rigor in early shipping**

"Let's get something out the door; we'll add the checks later." The checks are the platform. A "platform" shipped without the rigor gate, without the peer-review panel, without calibration tracking, without the ledger, is not the platform's MVP — it is a different product entirely. Phase 0 of Section 7 is small precisely so that the architectural commitments arrive on day one.

## 10. Glossary

**Adversarial generation**
The construction or retrieval of the strongest counter-claims to a candidate conclusion, run as part of the standard pipeline rather than as an after-the-fact audit.

**Atomic claim**
A single proposition — one subject, one predicate, one modality, ideally under thirty tokens — which is the unit of storage and reasoning. Distinct from a paragraph, summary, or document.

**Audit ledger**
A strictly append-only, hash-chained, signed log of every state-changing operation in the platform. The basis of reconstructible history.

**Calibration**
The match between the confidences a method (or person) assigns and the empirical frequency of those confidences being correct. Measured by Brier score, log loss, reliability diagrams, sliced by domain and time.

**Cascade**
The directed acyclic graph of all claims and principles a given conclusion descends from, annotated with how load-bearing each ancestor is.

**Citation span**
The exact byte range, time range, or page-and-line locator of a claim within its source. Required, non-nullable.

**Coherence engine**
The composable multi-layer validation system that returns a coherence vector for any claim, principle, or conclusion. Reference layers: logical, NLI, probabilistic, geometric, information-theoretic, LLM.

**Coherence vector**
The structured score returned by the coherence engine, with one component per validation layer, deliberately never reduced to a single scalar by default.

**Conclusion**
A synthesized argued position, the platform's publication unit. Tracked through draft, review, blocked, published, and retracted states.

**Conviction**
A weighted measure of how strongly the platform takes a principle to be held by the organization, computed from claim frequency, emphasis, argumentative centrality, and speaker authority.

**Decay**
The process by which the freshness of a node ages and, if not reinforced, eventually retires the node from active reasoning while keeping it queryable.

**Drift**
A method's silent regression: its output on a frozen test set diverges from its baseline. Detected by scheduled drift-monitor jobs.

**External battery**
The set of adapters that periodically tests the platform's claims and methods against external

benchmarks — prediction markets, fact-checking corpora, replication datasets.

**Forecast**
A future-resolvable probabilistic claim, with an unambiguous resolution criterion and a date.

**Forward inference**
The path from question to reasoned answer using the existing graph.

**Geometric layer**
The coherence layer that uses learned contradiction directions and difference-vector analysis in embedding space, rather than raw cosine similarity, which is insufficient (the "cosine paradox").

**Ingestion guard**
The pre-extraction filter that rejects malicious, spam, prompt-injecting, or quota-violating inputs.

**Intellectual Capital Platform (ICP)**
The architecture this document specifies: a three-layer system that turns an organization's unstructured deliberation into a navigable, queryable, validated body of knowledge.

**Inverse inference**
The path from observed outcome back to the principles most consistent with that outcome. Used in post-mortems and revision.

**Method**
A first-class persisted algorithm with a declared interface, versioned implementation, drift baseline, and track record. The product of Invariant 7.

**Method invocation**
The recorded fact of a single call to a method, with input hash, output hash, duration, status, and tenant. The unit of method-level observability.

**Mitigation**
An always-on guard that protects against a specific known failure mode, distinct from a rigor-gate check (which fires only at publication).

**Peer-review panel**
The collection of independent reviewer methods that evaluate each conclusion from a single explicit perspective each, deliberately not aggregated into consensus.

**Principle**
A cluster of related atomic claims, treated as a single weighted unit with conviction, coherence vector, and drift history.

**Provenance**
The bidirectional traceability between source and conclusion. A non-optional invariant.

**Rigor gate**
The single chokepoint through which all candidate publications must pass, running a list of registered checks with pass / block / require-override outcomes.

**Tenancy**
The discipline of threading a `TenantContext` through every operation so the platform can serve multiple isolated organizations. Designed in from day one.

**Track record**
A method's structured history of outcomes, sliced by domain and time, the basis of calibration analysis and drift detection.

**Transfer**
The packaging, signing, dockerizing, and harnessing of a method bundle for export, so that the organization's methodology is itself a shippable product.

---

*End of specification.*